Факультет компьютерных наук

Системное программирование

Москва 2025

Статический анализ программ на языке Scala на основе унифицированного абстрактного синтаксического дерева

Static analysis of Scala programs based on the unified abstract syntax tree

Выполнил:
Пасилецкий Даниил Олегович

Руководитель:
Профессор Базовая кафедра «Системное программирование» ИСП РАН, факультета компьютерных наук,
Белеванцев Андрей Андреевич

Консультант:
Стажер-исследователь ИСП РАН,
Афанасьев Виталий Олегович

Scala

```scala
sealed trait Shape
case class Circle(r: Double) extends Shape
case class Rect(w: Double, h: Double) extends Shape

def area(s: Shape): Double = s match
  case Circle(r) => math.Pi * r * r
  case Rect(w,h) => w * h

val total = List(Circle(2), Rect(3,4))
                .map(area)
                .sum
```

Листинг 1. Пример кода на Scala

```java
interface Shape {
   double area();
}

class Circle implements Shape {
    double r;
    Circle(double r){
       this.r = r;
    }
    public double area(){
       return Math.PI*r*r;
    }
}

class Rect implements Shape {
    double w, h;
    Rect(double w, double h){
       this.w=w;
       this.h=h;
    }
    public double area(){
      return w * h;
    }
}

double total = 0;
for (Shape s : shapes)
    total += s.area();
```

Листинг 2. Пример кода на Java

Scala

```
1.  object Demo extends App {
2.    private lazy val x: Int = x + 1
3.    println(x)
4.  }
```

Листинг 3. Lazy на Scala

Scala

```
1.  object Demo extends App {
2.    private lazy val x: Int = x + 1
3.    println(x)
4.  }
```

Листинг 3. Lazy на Scala

Scala

```
1.  object Demo extends App {
2.    private lazy val x: Int = x + 1
3.    println(x)
4.  }
```

Листинг 3. Lazy на Scala

java.lang.StackOverflowError

Scala

```
1.  object Demo extends App {
2.    private lazy val x: Int = x + 1
3.    println(x)
4.  }
```

Листинг 3. Lazy на Scala

java.lang.StackOverflowError

```
1.  public final class Demo$ implements scala.App {
2.      private static int x;
3.
4.      private static volatile boolean bitmap$0;
5.
6.      private int x$lzycompute() {
7.          synchronized (this) {
8.              if (!bitmap$0) {
9.                  x = x() + 1;
10.                 bitmap$0 = true;
11.             }
12.         }
13.         return x;
14.     }
15.
16.     public int x() {
17.         if (!bitmap$0) {
18.             return x$lzycompute();
19.         } else {
20.             return x;
21.         }
22.     }
23.
24.     public final void delayedEndpoint$Demo$1() {
25.         scala.Predef$.MODULE$.println(
26.             Integer.valueOf(x())
27.         );
28.     }
29. }
```

Листинг 4. Декомпилированный кода с lazy

Scala

```
1.  object Demo extends App {
2.    private lazy val x: Int = x + 1
3.    println(x)
4.  }
```

Листинг 3. Lazy на Scala

java.lang.StackOverflowError

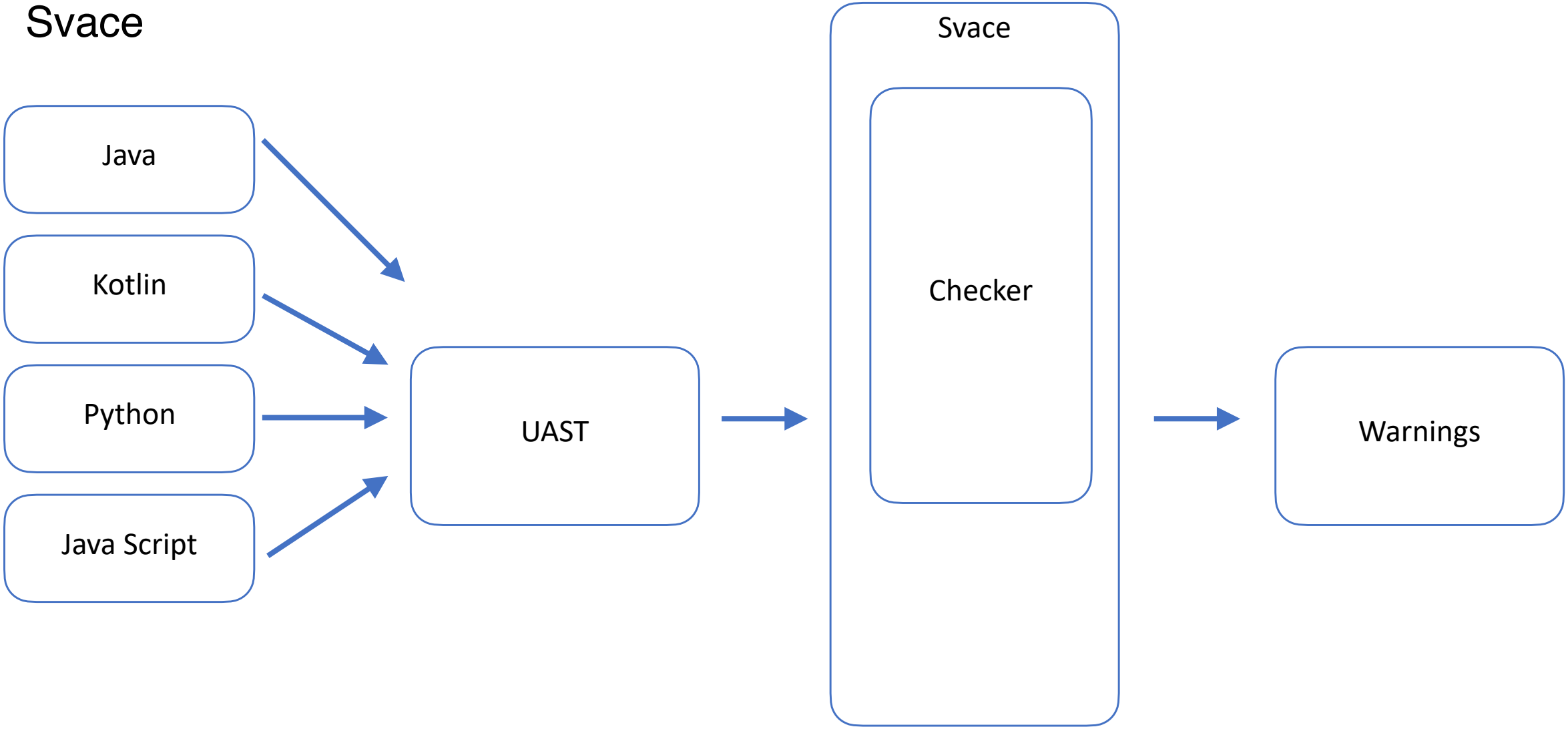
```
1.  public final class Demo$ implements scala.App {
2.      private static int x;
3.
4.      private static volatile boolean bitmap$0;
5.
6.      private int x$lzycompute() {
7.          synchronized (this) {
8.              if (!bitmap$0) {
9.                  x = x() + 1;
10.                 bitmap$0 = true;
11.             }
12.         }
13.         return x;
14.     }
15.
16.     public int x() {
17.         if (!bitmap$0) {
18.             return x$lzycompute();
19.         } else {
20.             return x;
21.         }
22.     }
23.
24.     public final void delayedEndpoint$Demo$1() {
25.         scala.Predef$.MODULE$.println(
26.             Integer.valueOf(x())
27.         );
28.     }
29. }
```

Листинг 4. Декомпилированный кода с lazy

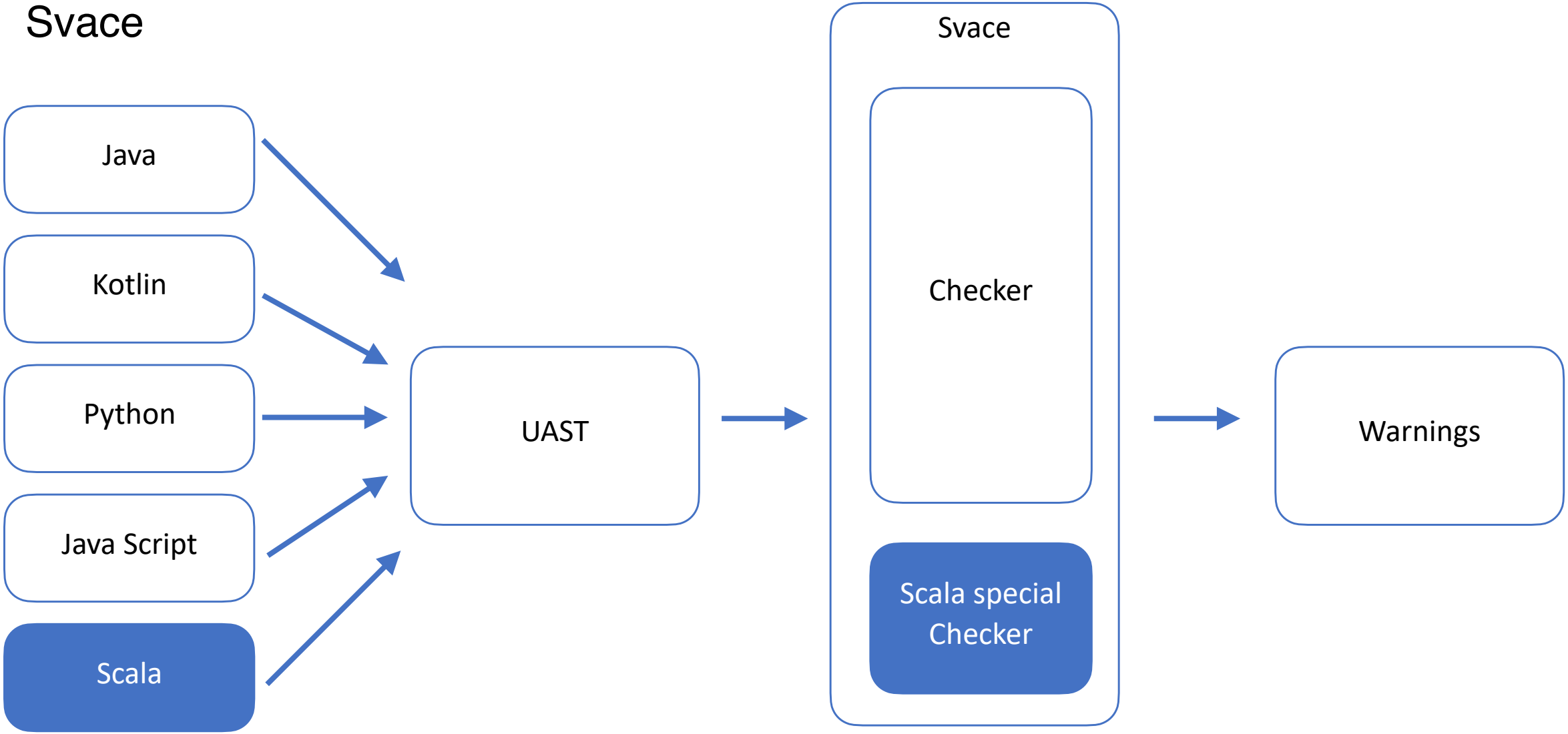
Проблема

В анализаторе Svace уже есть поддержка языка Scala, но она работает только на уровне **байткода JVM**, что не позволяет обнаруживать ряд ошибок, которые **явно видны только в исходном коде**.

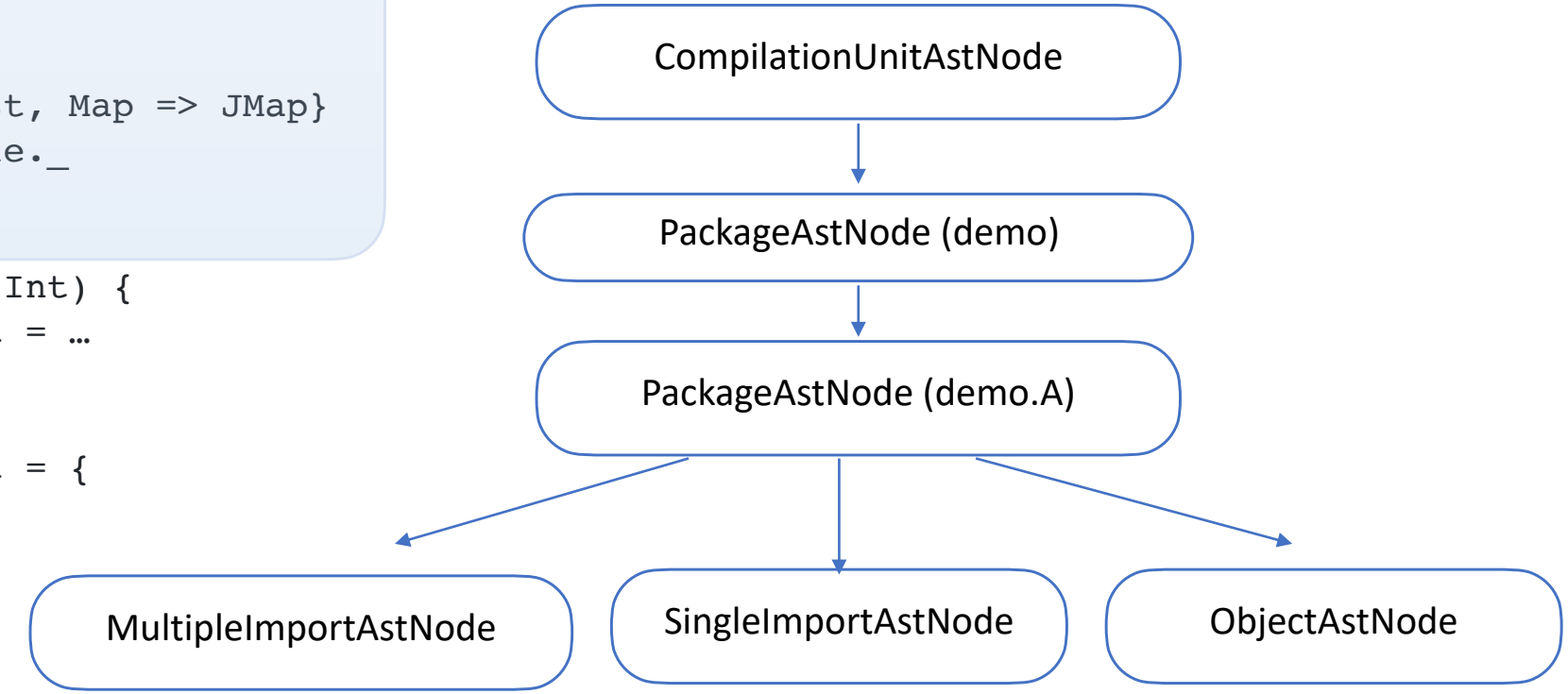
Svace


Java
Kotlin
Python
Java Script
UAST
Svace
Checker
Warnings

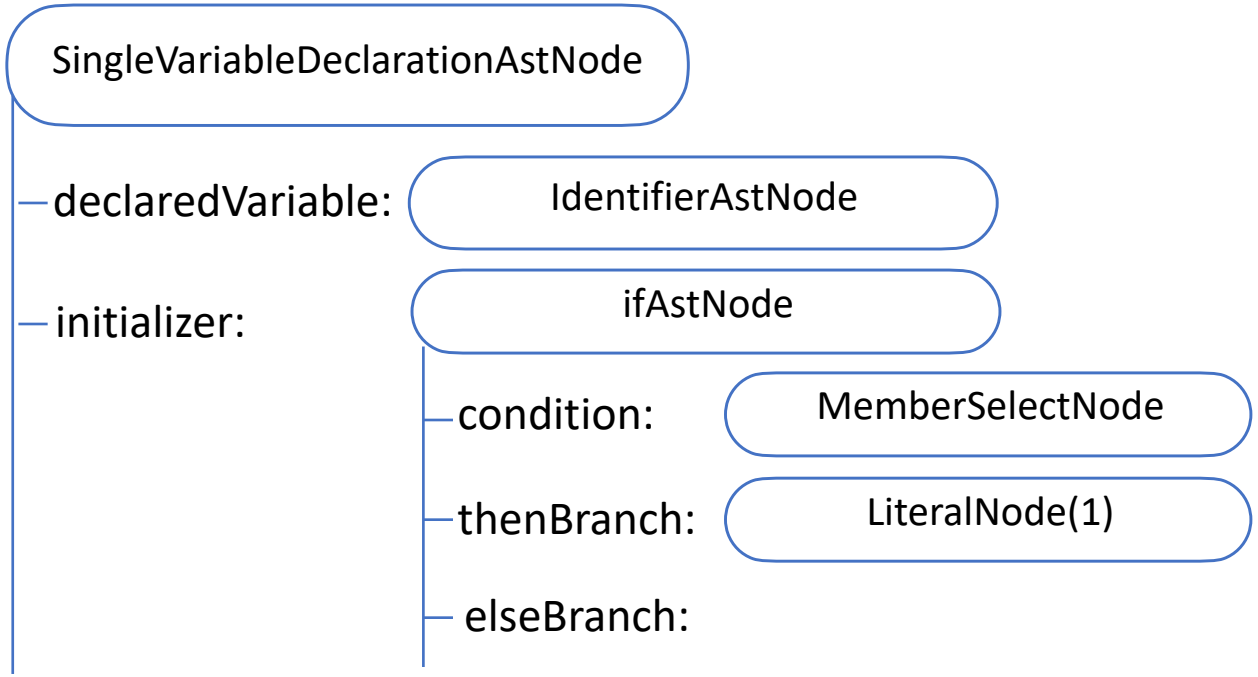
Svace


Java
Kotlin
Python
Java Script
Scala
UAST
Svace
Checker
Scala special Checker
Warnings

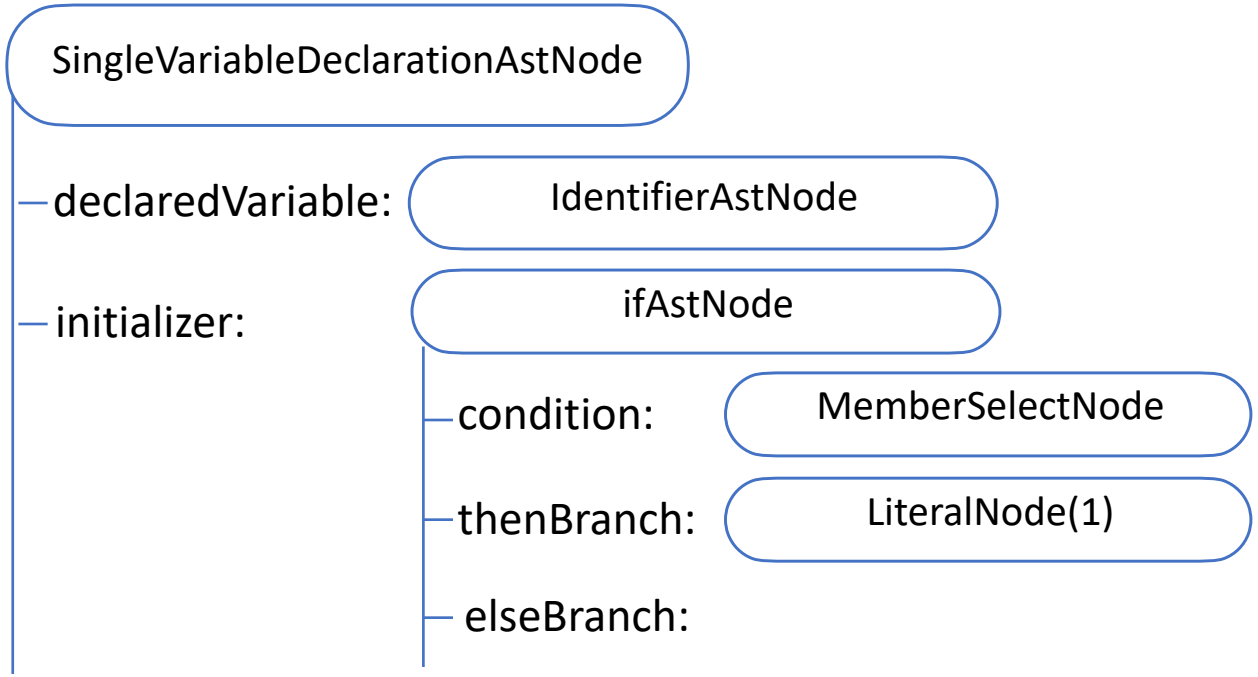
```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def +(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty) 1
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.    } {
19.       println(p.x)
20.    }
21.     val sum = Point(1, 0) + Point(2, 0) + Point(3, 0)
22.   }
23. }
```



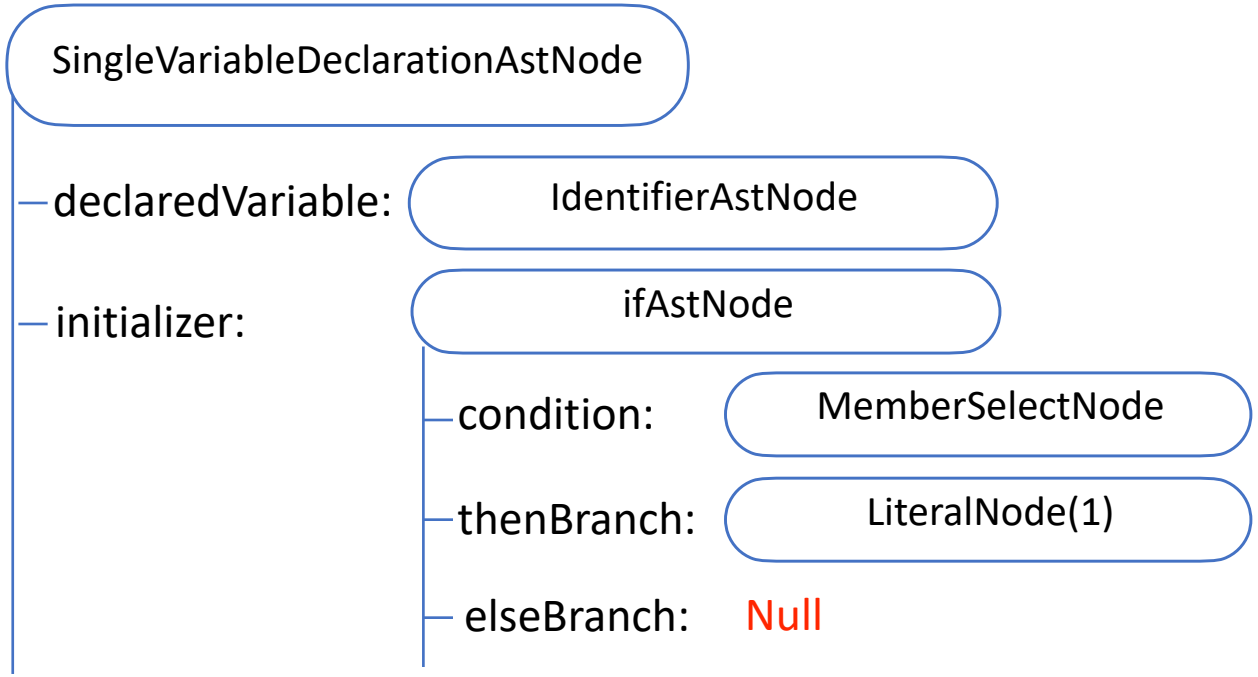
```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def +(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty) 1
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.    } {
19.       println(p.x)
20.    }
21.     val sum = Point(1, 0) + Point(2, 0) + Point(3, 0)
22.   }
23. }
```



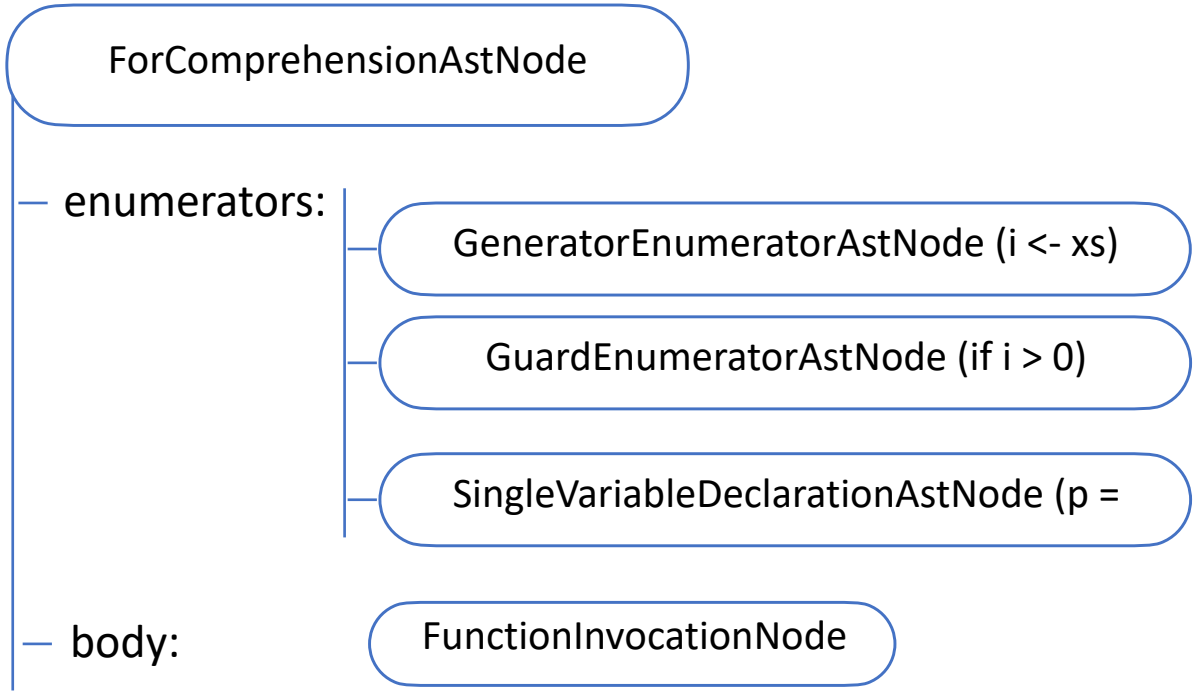
```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def +(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty) 1 else ()
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.    } {
19.       println(p.x)
20.    }
21.     val sum = Point(1, 0) + Point(2, 0) + Point(3, 0)
22.   }
23. }
```



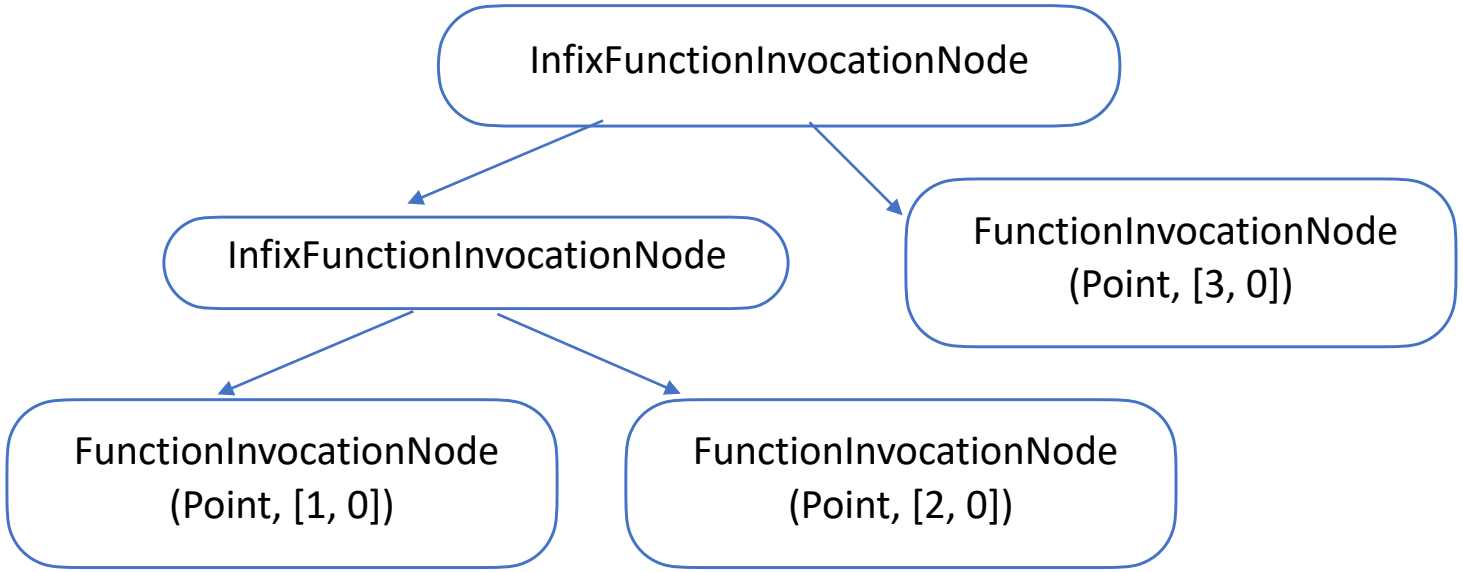
```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def +(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty) 1
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.    } {
19.       println(p.x)
20.    }
21.     val sum = Point(1, 0) + Point(2, 0) + Point(3, 0)
22.   }
23. }
```



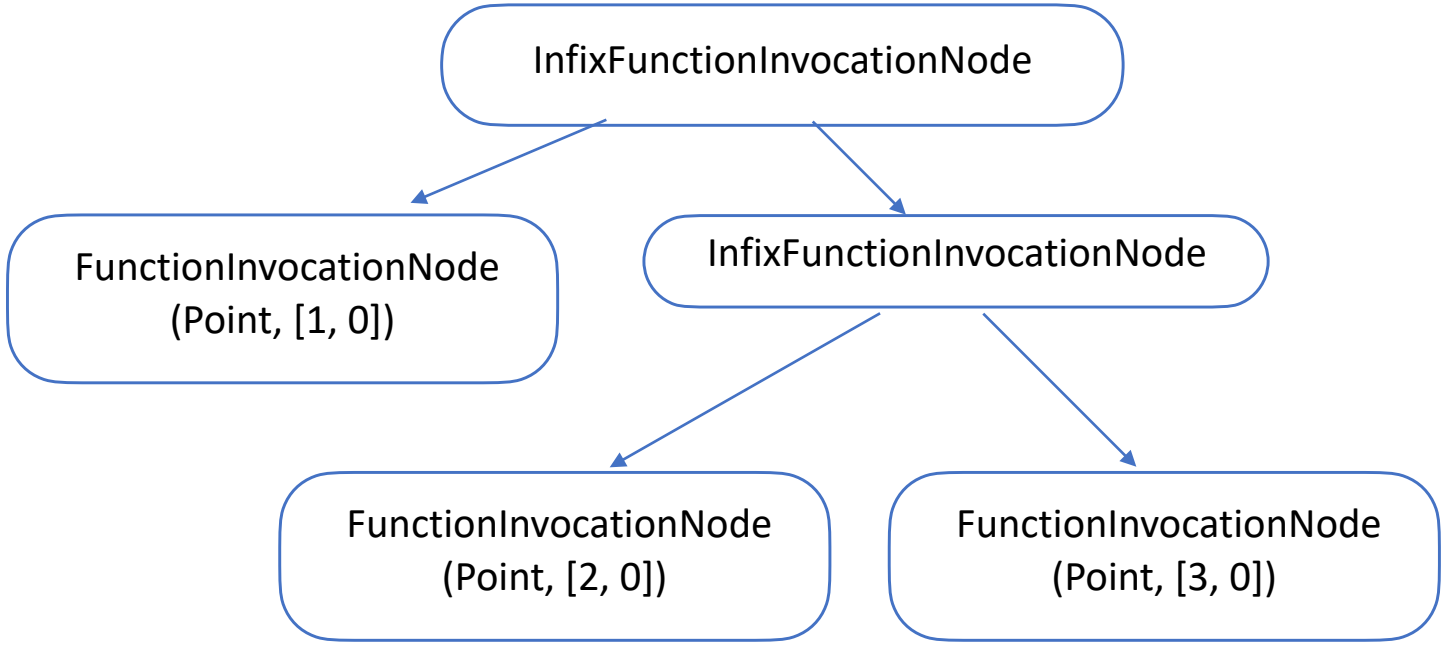
```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def +(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty) 1
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.     } {
19.       println(p.x)
20.     }
21.     val sum = Point(1, 0) + Point(2, 0) + Point(3, 0)
22.   }
23. }
```



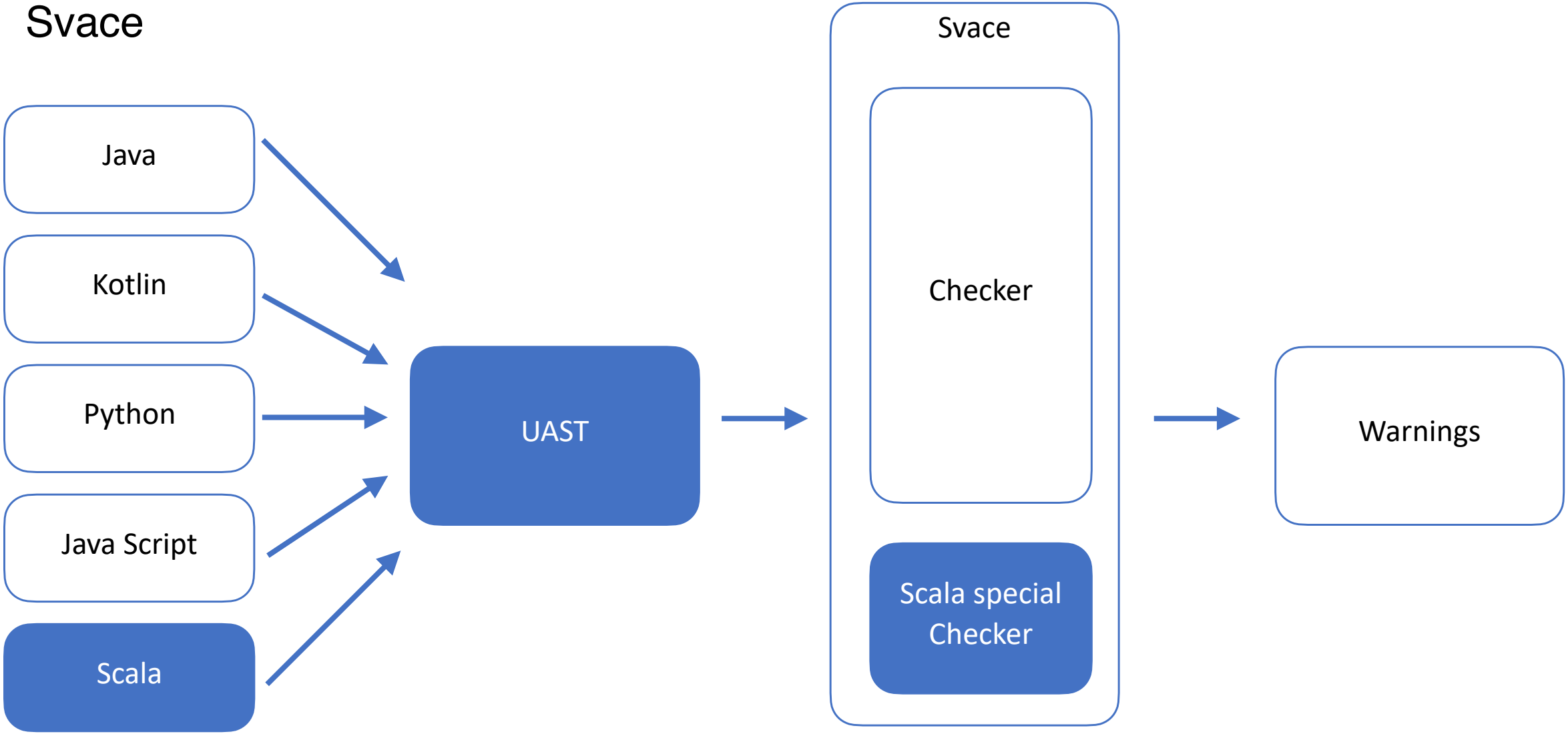
```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def +(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty)
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.    } {
19.       println(p.x)
20.    }
21.     val sum = Point(1, 0) + Point(2, 0) + Point(3, 0)
22.   }
23. }
```



```
1.  package demo
2.  package A
3.
4.  import java.util.{List => JList, Map => JMap}
5.  import scala.collection.mutable._
6.
7.  object Example {
8.    case class Point(x: Int, y: Int) {
9.      def :+(other: Point): Point = …
10.   }
11.
12.   def run(xs: List[Int]): Unit = {
13.     val a = if (xs.nonEmpty)
14.     for {
15.       i <- xs;
16.       if i > 0;
17.       p = Point(i, i * 2)
18.    } {
19.       println(p.x)
20.    }
21.     val sum = Point(1, 0) :+ Point(2, 0) :+ Point(3, 0)
22.   }
23. }
```



Svace


Java
Kotlin
Python
Java Script
Scala
UAST
Svace
Checker
Scala special Checker
Warnings

Постановка задачи

Цель: Добавить поддержку анализа исходного кода Scala на основе абстрактного синтаксического дерева в систему статического анализа Svace.

Для достижение этой цели необходимо выполнить следующие задачи:

1. Разработать транслятор из исходного кода на языке Scala (версии 2.0) в унифицированное абстрактное синтаксическое дерево;
2. Разработать Scala специфичные чекеры;
3. Провести экспериментальную оценку результата

Оценка результатов

Набор данных:

Открытые проекты написанные на Scala 2.

Объём: несколько сотен тысяч строк кода.

Apache Spark - https://github.com/apache/spark

Apache Kafke - https://github.com/apache/kafka

Сценарий анализа

Запускаем стандартный анализ Scala на уровне JVM байткода

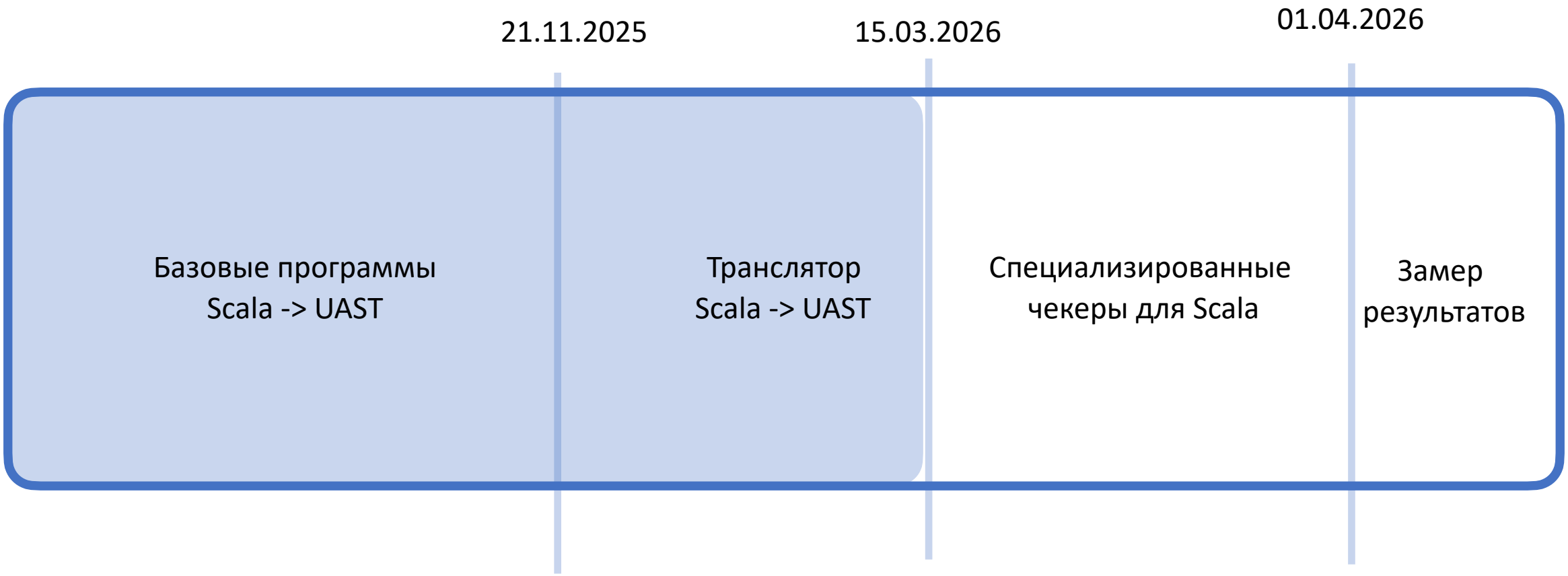
И запускаем анализ с AST модулем

Метрики

Количество найденных ошибок

Время анализа

План работ


21.11.2025
15.03.2026
01.04.2026
Базовые программы Scala -> UAST
Транслятор Scala -> UAST
Специализированные чекеры для Scala
Замер результатов

Факультет компьютерных наук

Системное программирование

Москва 2025

Статический анализ программ на языке Scala на основе унифицированного абстрактного синтаксического дерева

Static analysis of Scala programs based on the unified abstract syntax tree

Выполнил:
Пасилецкий Даниил Олегович

Руководитель:
Профессор Базовая кафедра «Системное программирование» ИСП РАН, факультета компьютерных наук,
Белеванцев Андрей Андреевич

Консультант:
Стажер-исследователь ИСП РАН,
Афанасьев Виталий Олегович